consumer. Complexity and orchestration like this are what make the difference in using a tool for CI or CD.

**Moving to continuous delivery**

Although most software projects have CI, getting your team to CD is first a cultural shift. Having your entire team understand and accept that small changes will be deployed to your consumer automatically and frequently can be a bit scary for teams that release even as frequently as every few weeks. Once you come to grips with the fact that you can change your consumer's experience multiple times a day, you're likely going to have to make significant changes to your build systems to accommodate the types of automation you'll need. You'll also likely need to make big changes in how you think about testing, because all testing in a CD world should rely on trustworthy automation rather than manual button pushing. What should be obvious (but rarely is) is to get the most out of CD you should have a good sense of why you're doing it and how you're monitoring the success of your changes. Fortunately, if you're implementing what you're learning in this book, monitoring should be the easy part.

### 5.1.3. Continuous testing

Continuous testing (CT) is also a part of CI; it's the practice of having tests continuously running on your codebase as you make changes to it. Teams that take the time to automate their testing process, write tests while they're writing their code, and run those tests constantly throughout the build process and on their local development environments end up with another rich set of data to use in the continuous improvement process.

To clarify where this testing happens and what this testing is, we take another look at figure 5.5 but highlight the testing in figure 5.6.

**Figure 5.6. Continuous testing in the scope of CI/CD**